

Chapter 5 - System Design

5.1 Introduction	1
5.2 System Architecture Design.....	2
Software Architecture Diagram	2
5.3 Class Diagram.....	3
UML Class Diagram.....	4
5.4 Use Case Diagram	4
5.5 Database Design	6
Entity Relationship Diagram (ERD).....	7
5.6 User Interface (UI) Design.....	7
5.7 Summary	13

5.1 Introduction

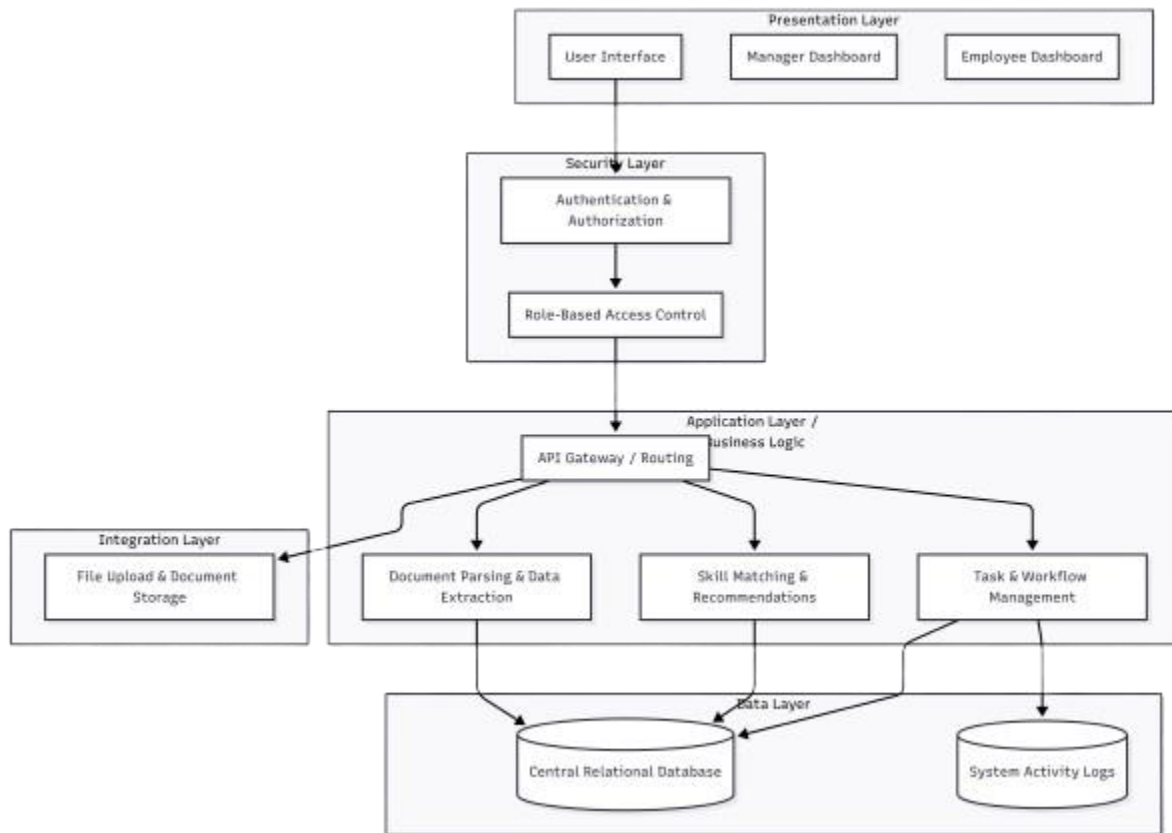
This chapter converts the specifications outlined in Chapter 4 into a concrete architectural and technical design. It defines the structure, components, data flow, and interaction of the Intelligent Task Allocation System (ITAS) to guide the implementation phase. By translating the functional and non-functional requirements into a logical blueprint, this chapter ensures that the system is modular, scalable, and capable of executing its core objectives, such as NLP-based skill extraction and algorithmic task matching.

5.2 System Architecture Design

The ITAS follows a layered, service-oriented architecture suitable for web applications. The architecture is logically divided into distinct layers to ensure modularity and maintainability, utilizing the technology stack identified in the methodology phase.

- **Presentation Layer:** This is the front-end user interface built with React.js. It handles user interactions, rendering dynamic dashboards for Project Managers and Employees, and displaying charts for workload and task progress.
- **Application Layer:** The core backend built using Python (Django or Flask). It contains the business logic, including the routing, task management workflows, and the Suitability Scoring algorithms. This layer also houses the NLP Engine (utilizing spaCy/NLTK and PyPDF2) for document parsing and skill extraction.
- **Data Layer:** Utilizes PostgreSQL to securely store and manage relational data. It manages structured data such as user profiles, tasks, extracted skill tags, and audit logs.
- **Integration Layer:** Manages internal file handling (CV and portfolio uploads in PDF format) and any potential external API connections (e.g., GitHub) for future portfolio parsing.
- **Security Layer:** Implements Role-Based Access Control (RBAC) separating Project Managers and Employees. It handles secure authentication, session management, and ensures that user credentials and sensitive data are encrypted.

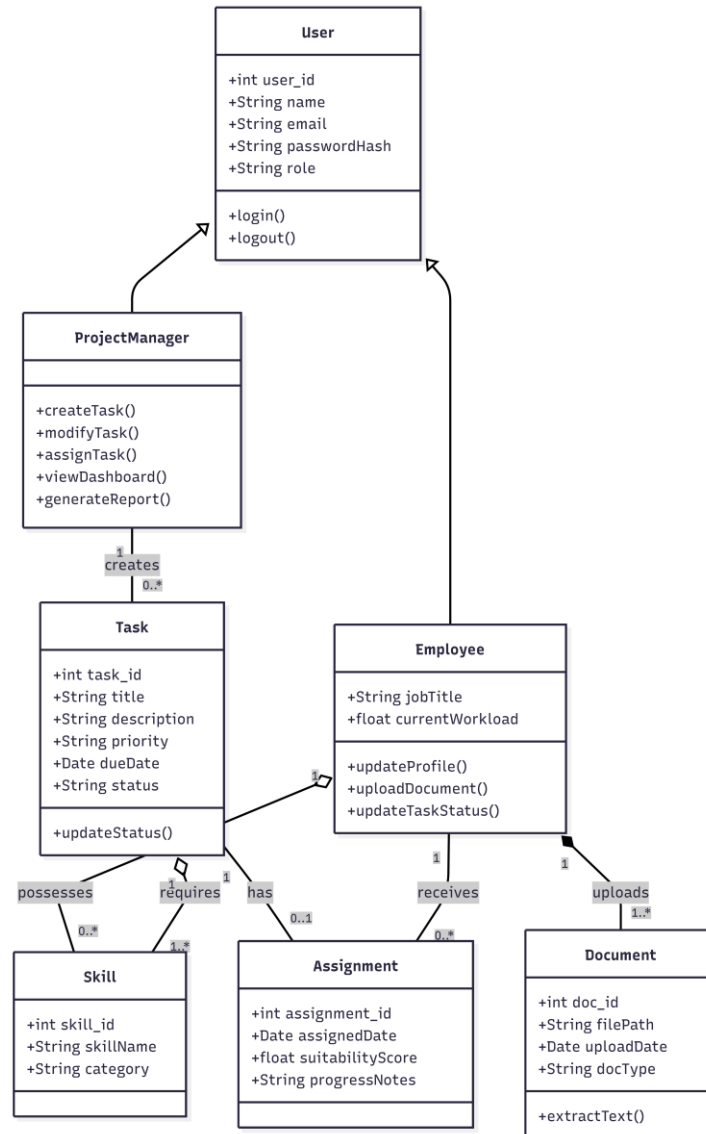
Software Architecture Diagram



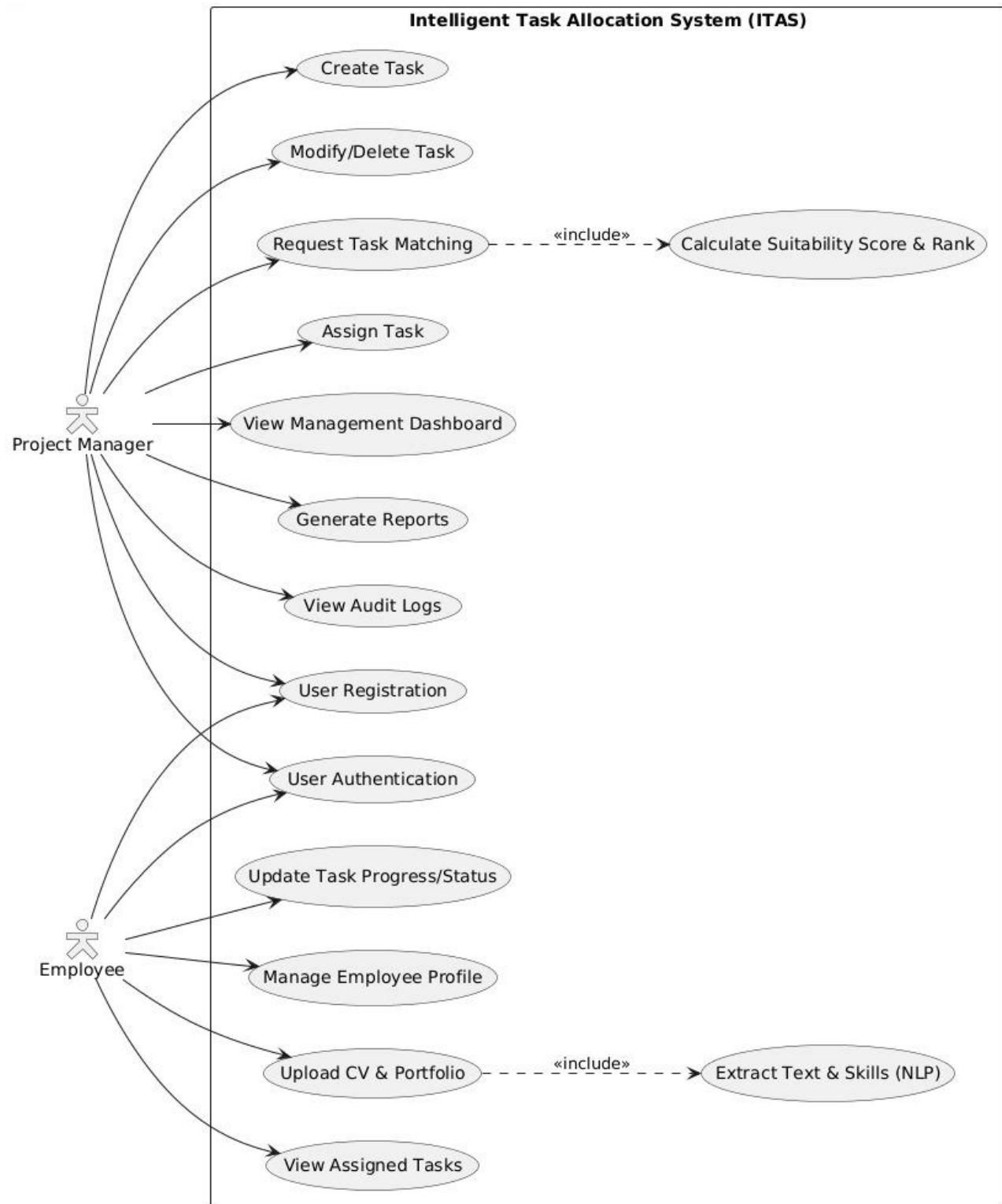
5.3 Class Diagram

The system's object-oriented structure is designed around the core entities required for task allocation. The class diagram outlines the attributes and operations of Users, Tasks, Skills, and Documents.

UML Class Diagram



5.4 Use Case Diagram

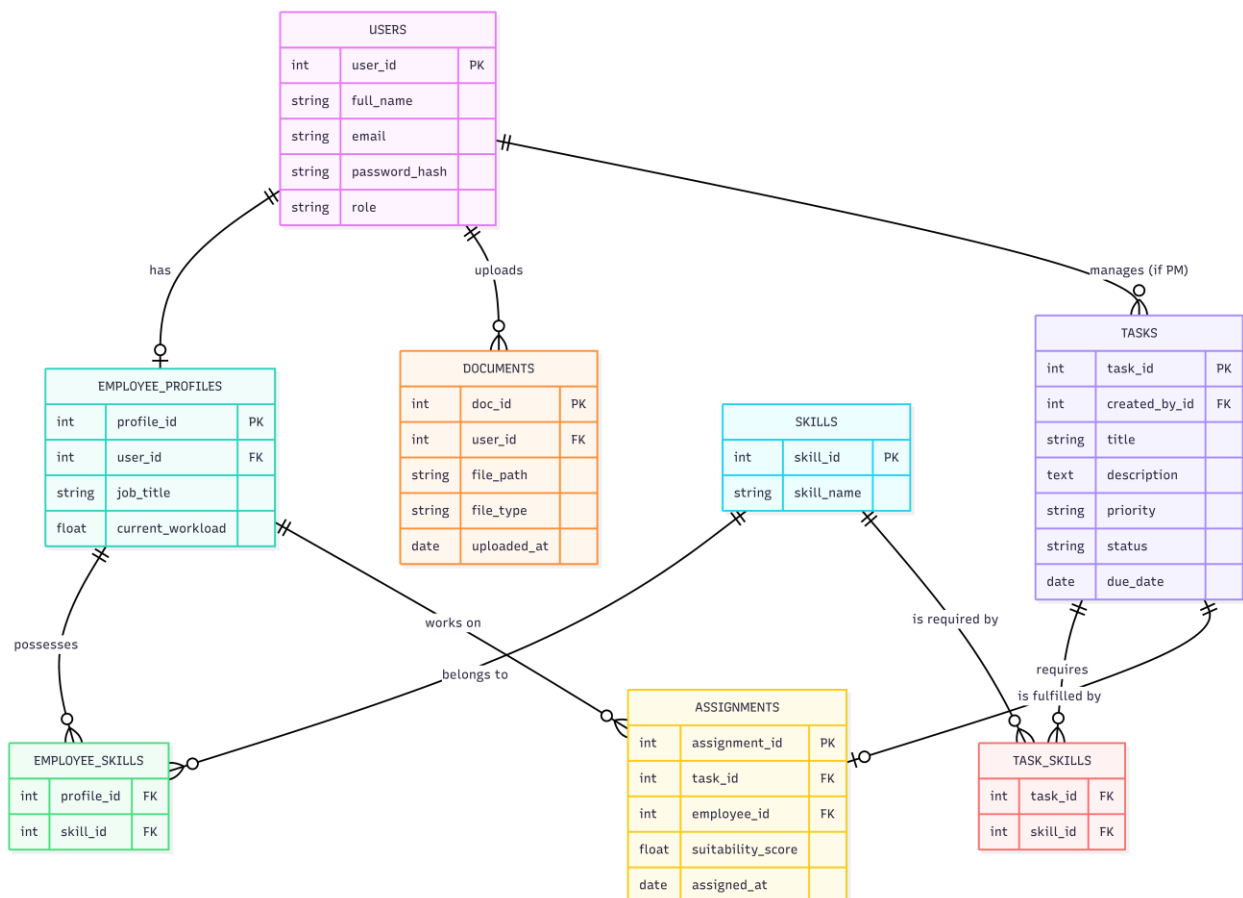


5.5 Database Design

The relational database is built using PostgreSQL. The schema is highly normalized to prevent data redundancy and ensure fast querying during the task-matching process.

- **Users Table:** Stores credentials and distinguishes roles (PM vs. Employee).
- **Tasks Table:** Stores task metadata (title, due date, status workflow: To Do, In Progress, Blocked, Completed, Cancelled).
- **Skills Table:** A centralized repository of extracted technical skills.
- **Junction Tables (Task_Skills & Employee_Skills):** Resolves many-to-many relationships, allowing the NLP engine to map multiple skills to a single employee and the PM to attach multiple skill tags to a task.
- **Assignments Table:** Links an Employee to a Task, storing the calculated suitability score and timestamps.

Entity Relationship Diagram (ERD)



5.6 User Interface (UI) Design

The User Interface of the Intelligent Task Allocation System (ITAS) is designed to be modern, intuitive, and highly responsive, built using React.js. The design prioritizes data visualization and workflow efficiency, ensuring that both Project Managers (PMs) and Employees can navigate complex information—such as task queues, skill matrices, and workloads—with minimal cognitive load.

Layout Principles: The application utilizes a standardized dashboard layout comprising three main structural elements:

1. **Left Sidebar Navigation:** A persistent navigation menu tailored to the user's role. It displays the active workspace module and a quick-access user profile/sign-out block at the bottom.
2. **Top Header (Command Center):** Features a global search bar ("Search tasks, people, or skills") and a primary call-to-action button (e.g., "New Task"), ensuring core functions are always accessible.
3. **Main Content Area:** Employs a modular, card-based design. Information is grouped into distinct, rounded containers, allowing users to scan key performance indicators (KPIs), lists, and forms easily.

Color Scheme: The system features a refined "Dark Mode" aesthetic (deep navy/charcoal backgrounds) that reduces eye strain during prolonged use.

- **Backgrounds:** Dark blue/gray tones (#1E293B or similar) to create depth.
- **Accents & Typography:** Crisp white and light gray text for readability.
- **Interactive Elements:** Bright cyan/blue for primary buttons, active tabs, and skill tags (e.g., "React", "Advanced SQL").
- **Status Indicators:** Semantic colors are used effectively—green for "Ready" statuses or low-risk metrics, and progress bars that visually contrast against the dark background.

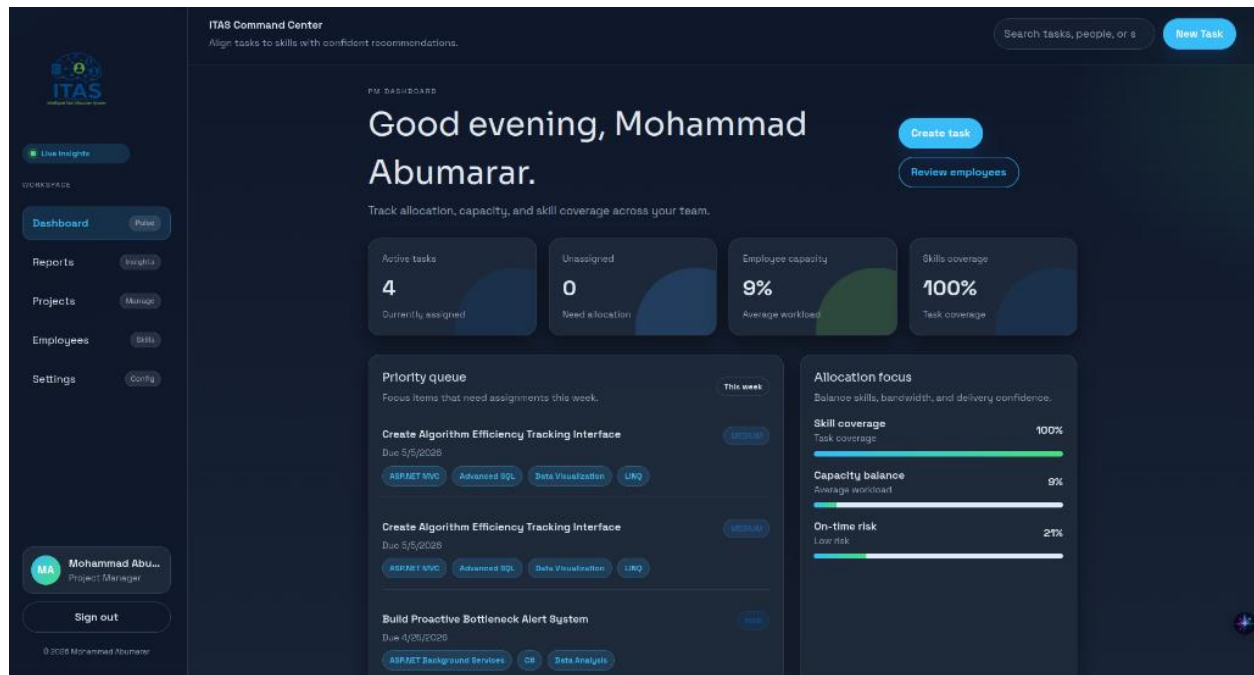
Navigation Structure (Role-Based): The UI dynamically adjusts based on the authenticated user's role:

- **Project Manager Workspace:** Includes **Dashboard** (pulse/overview), **Reports** (insights), **Projects** (manage initiatives), **Employees** (talent directory and skills), and **Settings**.

- **Employee Workspace:** Streamlined to focus on personal responsibilities, featuring **My Profile** (overview of skills, active tasks, and CV upload) and **Settings**.

Core Interface Walkthrough:

- **PM Dashboard:** Acts as the command center. It prominently displays quick KPIs (Active tasks, Unassigned, Employee capacity, Skills coverage). The "Priority queue" lists tasks needing assignment alongside their required skill tags (e.g., "ASP.NET MVC"), while the "Allocation focus" panel visualizes capacity balance and on-time risks.



- **Employees (Talent Directory):** Provides PMs with a comprehensive view of the team. Top summary cards track CV parsing statuses. Individual employee cards display their name, role, a visual cluster of extracted skills, their current workload percentage via a progress bar, and document management icons.

The screenshot displays the 'ITAS Command Center' interface, specifically the 'Talent Directory' section for 'Employees'. The interface is dark-themed with a sidebar on the left and a main content area.

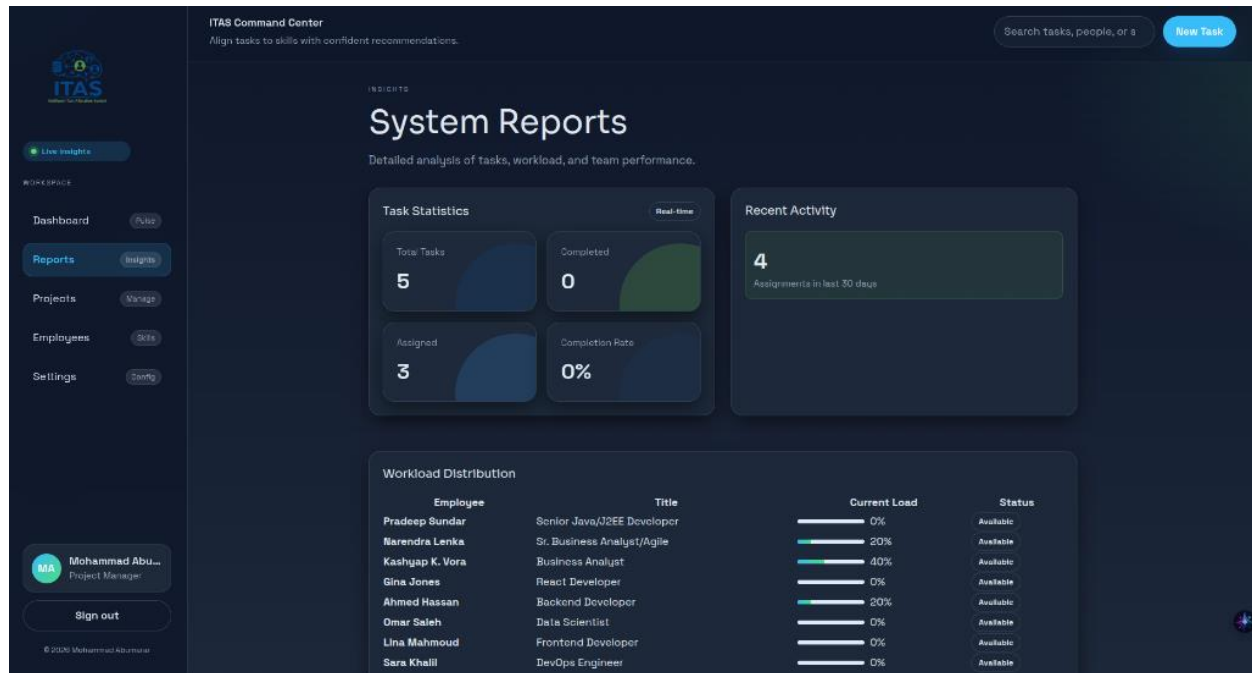
Left Sidebar:

- Logo: ITAS (Intelligent Talent Assessment System)
- Live Insights button
- Workspace section:
 - Dashboard (Full)
 - Reports (Insights)
 - Projects (Manage)
 - Employees (Skills)** (Active)
 - Settings (Config)
- User profile: Mohammad Abu... (Project Manager)
- Sign out button
- Footer: © 2020 Muhammad Aouass

Main Content Area:

- Header: ITAS Command Center, Align tasks to skills with confident recommendations. Search bar: Search tasks, people, or a. New Task button.
- Section: TALENT DIRECTORY, Employees. Search by name or email. Add Employee button.
- Sub-header: Upload CVs, review extracted skills, and keep profiles current.
- Summary Cards:
 - Total employees: 9 (Profiles in directory)
 - CVs ready: 9 (Parsed and available)
 - Processing: 0 (In extraction queue)
 - Missing CVs: 0 (Needs upload)
- Employee Cards (Grid):
 - Pradeep Sundar** (PS): Senior Java/JEE Developer. Skills: CSS, Java, JavaScript, JQuery, SQL, Spring, API, AWS. Last update: 3/26/2020, 1:47:34 AM. Workload: 100%. Upload CV button.
 - Narendra Lenka** (NL): Sr. Business Analyst/Agile. Skills: HTML, Agile, CSS, Design, Documentation, ETL, MySQL, SQL. Last update: 2/12/2020, 9:17:39 PM. Workload: 200%. Upload CV button.
 - Kashyap K. Vora** (KK): Business Analyst. Skills: Confluence, Design, Jira, Machine Learning, Oracle, R, SQL, Spark. Last update: 2/11/2020, 8:29:40 PM. Workload: 400%. Upload CV button.
 - Gina Jones** (GJ): Talent Developer. Status: Ready.
 - Ahmed Hassan** (AH): Talent Developer. Status: Ready.
 - Omar Saleh** (OS): Talent Developer. Status: Ready.

- **System Reports:** Offers detailed analytics. It features "Task Statistics" (completion rates) and a detailed "Workload Distribution" table, allowing PMs to see exactly who is available and who is nearing full capacity at a glance.



Workspace

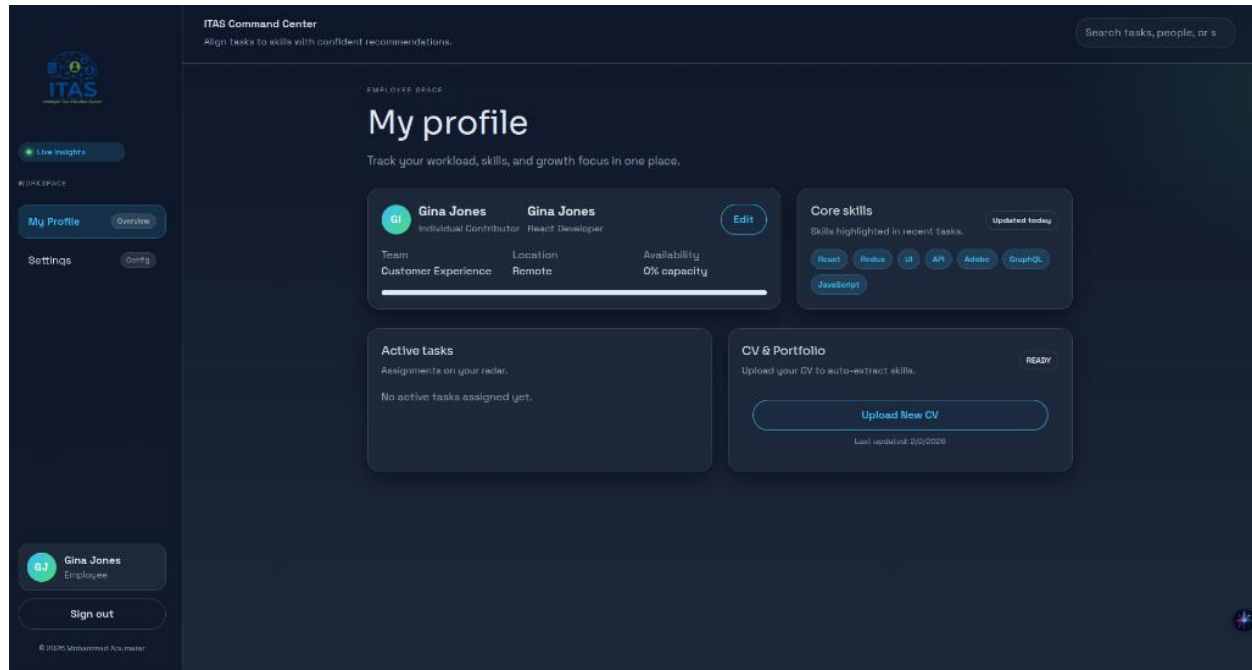
- Dashboard [Pulse](#)
- Reports [Insights](#)**
- Projects [Manage](#)
- Employees [Skills](#)
- Settings [Settings](#)

Mohammad Abu...
Project Manager

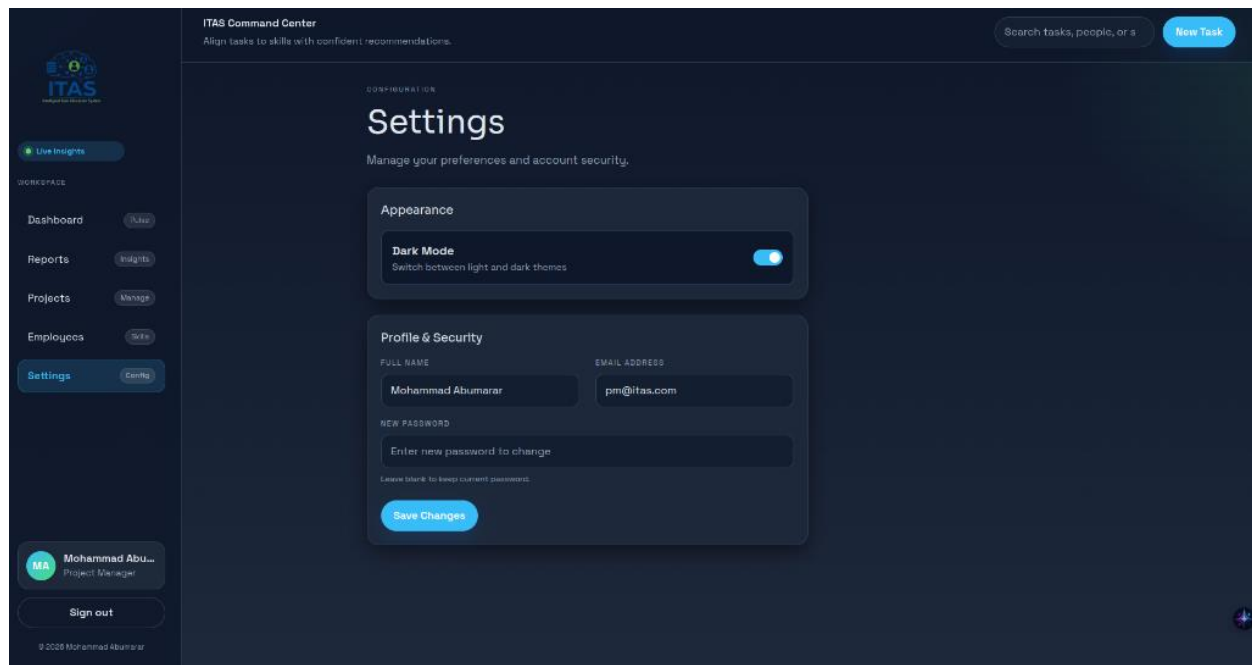
[Sign out](#)

© 2020 Mohammad Abu...

- **Employee Profile (Dashboard):** A personalized space for team members. It displays their "Core skills" (auto-extracted and highlighted), their "Active tasks," and a dedicated "CV & Portfolio" drop-zone for uploading their latest documents for the NLP engine to process.



- **Settings:** A clean, form-based interface for managing application appearance (Dark Mode toggle) and updating Profile & Security credentials.



5.7 Summary

The system design phase translates the functional and non-functional requirements of the ITAS into a concrete technical blueprint.

- **Efficiency:** The card-based React.js frontend ensures a snappy, single-page application experience, while the modular Python backend allows the NLP parsing and matching algorithms to run without blocking user interactions.
- **Security:** By implementing strict Role-Based Access Control (RBAC), the design ensures that sensitive managerial data (like cross-team workload and performance reports) is isolated from standard employee views. The application layer securely handles authentication and encrypted data transmission.
- **Scalability:** The normalized PostgreSQL database schema, combined with junction tables for dynamic skill mapping, ensures that as the company grows—adding more employees, projects, and highly varied skill tags—the database will not suffer from redundancy or performance degradation.
- **Maintainability:** The clear separation of concerns (Presentation, Application, Data, and Security layers) means that future enhancements, such as integrating third-party HR APIs or upgrading the NLP models, can be executed within the Application or Integration layers without requiring a complete overhaul of the User Interface.

Ultimately, this design serves as a robust and comprehensive guide for the implementation phase, ensuring the final product successfully addresses the "knowledge gap" and manual overhead inherent in traditional task allocation.